CprE 3810: Computer Organization and Assembly-Level Programming

Project Part 1 Report

Team Members: _Christian Lapnow & Jacob Dang_____

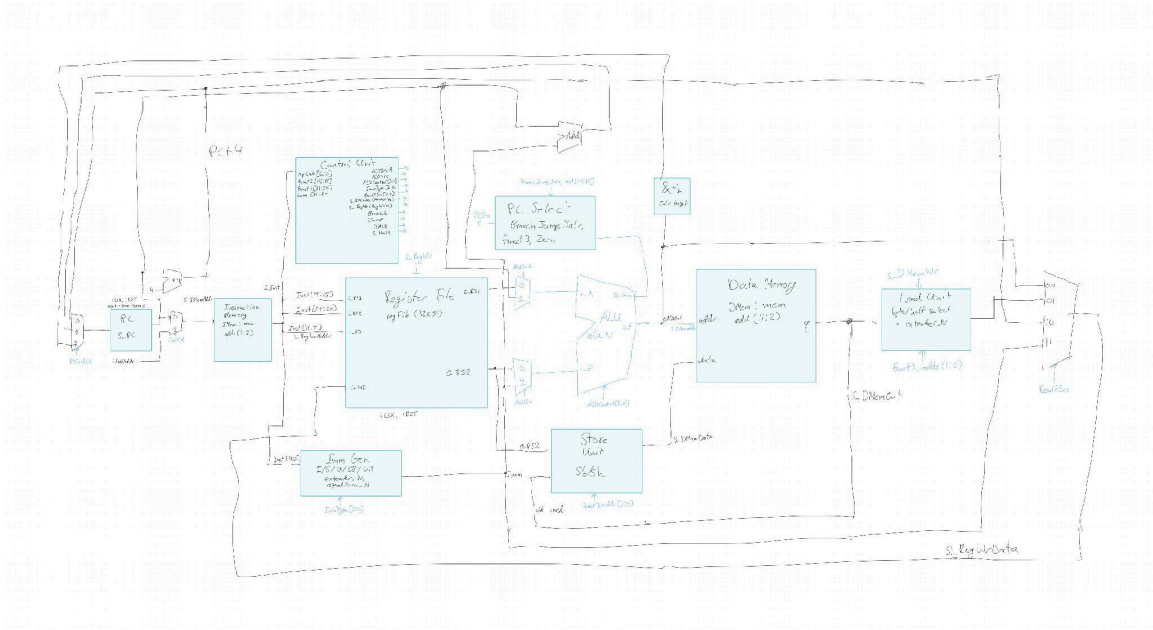
_____

_____

Project Teams Group #:___A_08_____

Refer to the highlighted language in the project 1 instruction for the context of the following questions.

[Part 2 (d)] Include your final RISC-V processor schematic in your lab report.



[Part 3.1.a.] Create a spreadsheet detailing the list of *M* instructions to be supported in your project alongside their binary opcodes and funct fields, if applicable. Create a separate column for each binary bit. Inside this spreadsheet, create a new column for the *N* control signals needed by your datapath implementation. The end result should be an *N***M* table where each row corresponds to the output of the control logic module for a given instruction.

Our spreadsheet (Proj1_control_signals.xlsx, included in the submission) has three tabs:

- Control Table: one row per instruction with its opcode, funct3, and funct7/immediate fields, the 11 control signal values, and a short description of what the instruction does. It covers all 37 required instructions, plus sltu (not

required, but implemented) and an extra addi row with inst[30] = 1 to show it must stay ADD.

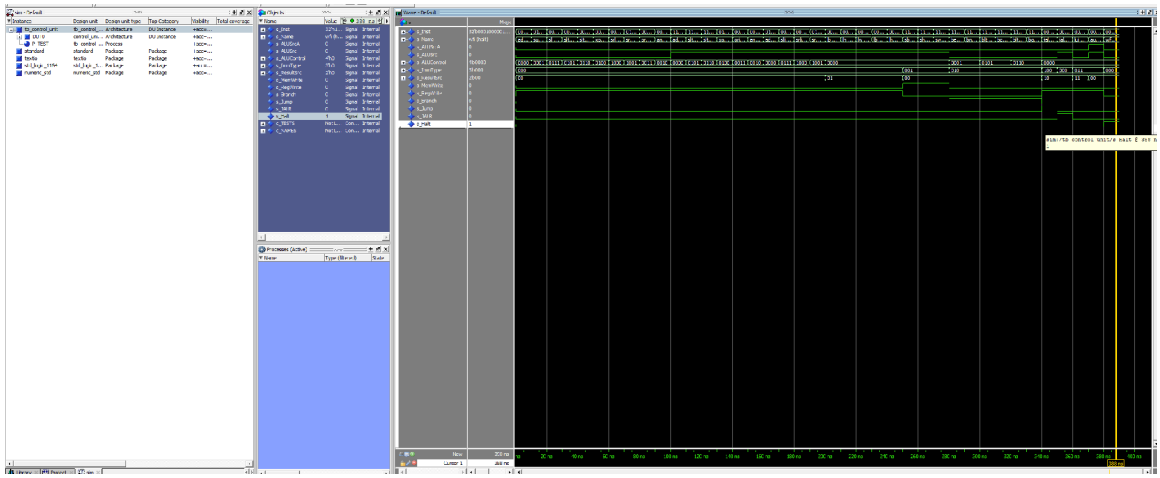
- Per-bit Table: the N*M table the assignment asks for. Each opcode, funct3, and funct7[5] bit has its own column, and every multi-bit signal is split into one column per bit (ALUControl[3:0], ImmType[2:0], ResultSrc[1:0]), for 28 columns per row.
- Signal Key: the width and encoding of each control signal, summarized below.

A "-" marks a don't care. The control unit drives 0 there and the testbench skips that field. funct3 is not a control unit output; it goes straight from the instruction to the fetch unit (branch type) and the load unit (size and sign).

Signal	**Width**	**Meaning**
ALUSrcA	1	0 = rs1, 1 = PC (auipc)
ALUSrc	1	0 = rs2, 1 = immediate
ALUControl	4	0000 ADD, 0001 SUB, 0010 AND, 0011 OR, 0100 XOR, 0101 SLT, 0110 SLTU, 0111 SLL, 1000 SRL, 1001 SRA
ImmType	3	000 I, 001 S, 010 B, 011 U, 100 J
ResultSrc	2	00 ALU, 01 load unit, 10 PC+4 (jal/jalr), 11 imm (lui)
MemWrite (s_DMemWr)	1	Write data memory (sw)
RegWrite (s_RegWr)	1	Write register file
Branch	1	Conditional branch instruction
Jump	1	jal
JALR	1	jalr (next PC comes from the ALU)
Halt (s_Halt)	1	wfi: opcode 1110011, funct3 000, imm 0x105

a

[Part 3.1.(b)] Implement the control logic module using whatever method and coding style you prefer. Create a testbench to test this module individually and show that your output matches the expected control signals from problem 1(a).



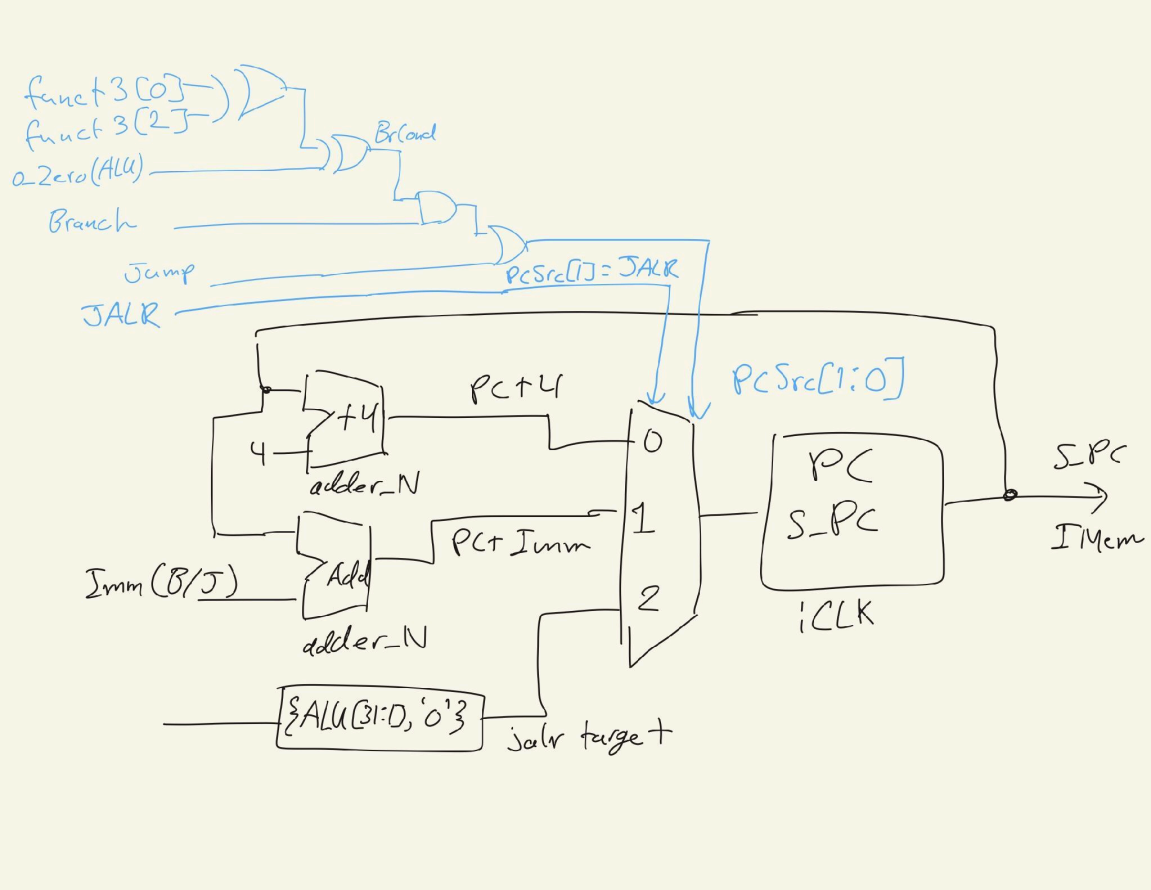
[Part 3.2. (a)] What are the control flow possibilities that your instruction fetch logic must support? Describe these possibilities as a function of the different control flow-related instructions you are required to implement.

The next PC is one of three values:

1. PC + 4: every non-control-flow instruction, and any conditional branch whose condition is false.
2. PC + imm: a taken branch (beq, bne, blt, bge, bltu, bgeu, using the B-type immediate) or jal (J-type immediate). Branches take this path only when their condition is true; jal always takes it.
3. (rs1 + imm) with bit 0 cleared: jalr. The ALU already computes rs1 + imm, so fetch takes the ALU output and forces bit 0 to 0.

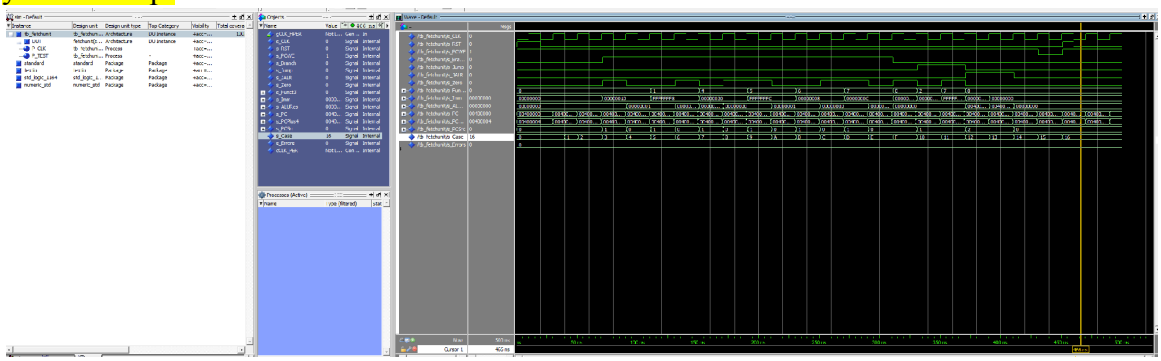
jal and jalr also write PC + 4 into rd (ResultSrc = 10). wfi ends the simulation through s_Halt rather than changing the PC.

[Part 3.2. (b)] Draw a schematic for the instruction fetch logic and any other datapath modifications needed for control flow instructions. What additional control signals are needed?



The figure shows our fetch logic. The PC register (s_PC) feeds IMem and two adder_N instances, one computing PC + 4 and one computing PC + imm (B-type for branches, J-type for jal). A 3:1 mux selected by PCSrc[1:0] chooses the next PC: 0 = PC + 4, 1 = PC + imm, 2 = the jalr target {ALU[31:1], '0'}. In VHDL this mux is two cascaded mux2t1_N, the first on PCSrc[0] and the second on PCSrc[1]. PCSrc = 11 never occurs because jalr sets Jump = 0. The PC register loads the mux output on iCLK and resets to 0x00400000 on iRST.

[Part 3.2.(c)] Implement your new instruction fetch logic using VHDL. Use QuestaSim to test your design thoroughly to make sure it is working as expected. Describe how the execution of the control flow possibilities corresponds to the QuestaSim waveforms in your writeup.

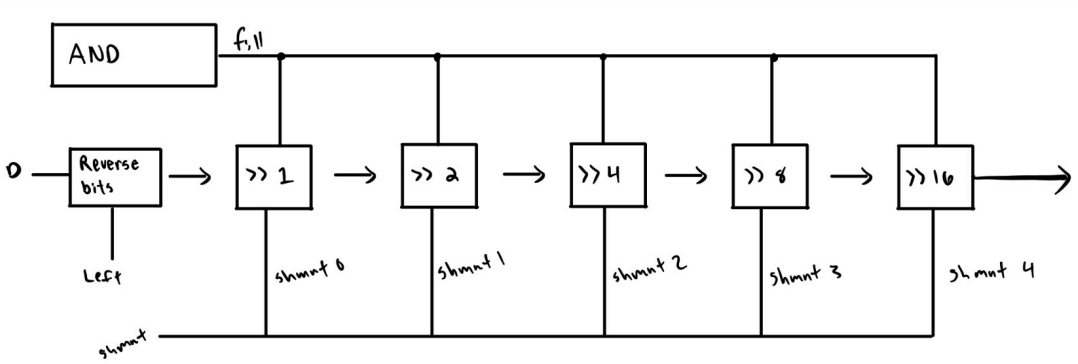


tb_fetchUnit.vhd runs 19 cycles that cover every control flow case the fetch logic must handle: reset, sequential execution, all six branch types both taken and not taken (with positive and negative offsets), jal forward and backward, jalr to an odd target, and an asynchronous reset mid-run. The testbench compares the PC each cycle against the expected value and reported 0 errors.
The waveform shows each case:

- **Sequential:** Branch, Jump, and JALR are all 0, so PCSrc = 00 and the PC advances by 4 each cycle.
- **beq taken:** Branch = 1 and Zero = 1 with funct3 = 000, so BrCond = 1 and PCSrc = 01. The PC jumps from 0x00400008 to 0x00400018 (+16).
- **Branch not taken:** Branch = 1 but BrCond = 0, so PCSrc stays 00 and the PC only advances by 4.
- **blt/bge/bltu/bgeu:** the ALU result is from slt/sltu, so Zero = 0 means "less than." The funct3 bits flip the condition, and the PC takes or skips the branch the same way as beq/bne.
- **jal backward:** Jump = 1 forces PCSrc = 01 regardless of Zero, and the PC moves back by 320.
- **jalr:** JALR = 1 gives PCSrc = 10, selecting the jalr target. ALU result 0x00400101 becomes PC 0x00400100 because bit 0 is cleared.
- **Reset:** asserting iRST mid-run sends the PC back to 0x00400000 immediately, without waiting for a clock edge.

[Part 3.3.1.(a)] Describe the difference between logical (`srl`) and arithmetic (`sra`) shifts. Why does RISC-V not have a `sla` instruction?
The srl instruction fills the upper bits with 0, while sra fills them with the sign bit, so a negative number stays negative. RISC-V has no sla because a left shift fills the low bits with 0 either way, so it would be the same as sll.

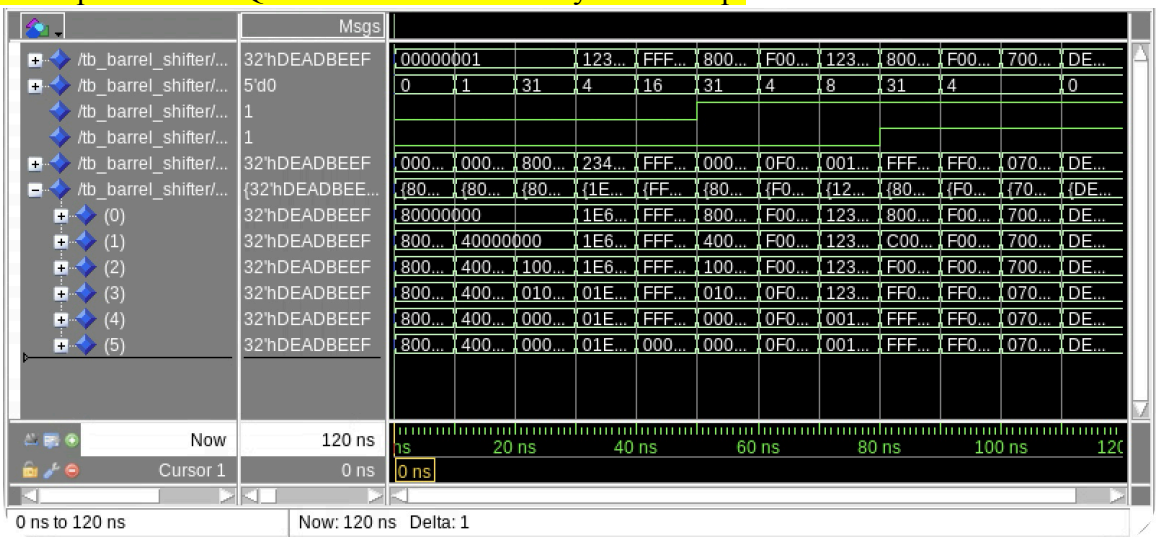
[Part 3.3.1.(b)] In your writeup, briefly describe how your VHDL code implements both the arithmetic and logical shifting operations.
The shifter (barrel_shifter.vhd) is five stages of 32 one-bit mux2t1s, written with generate loops. Stage k shifts right by 2^k when i_Shamt(k) is 1 and passes the word through otherwise, so any amount from 0 to 31 is the sum of the stages that are on (1, 2, 4, 8, 16). The muxes at the top of each stage that have nothing to shift in get the fill bit s_Fill = i_Arith and i_Right and i_D(31): 0 for srl, the sign bit for sra. The ALU drives i_Right from ALUControl(3) and i_Arith from ALUControl(0), so SRL (1000) and SRA (1001) differ only in the fill. No shift_right or other library functions are used.



[Part 3.3.1.(c)] In your writeup, explain how the right barrel shifter above can be enhanced to also support left shifting operations.

A left shift is a right shift of the bit-reversed word. One row of 32 mux2t1s (G_IN_REV) reverses the input when s_Left = not i_Right, the same five right-shift stages run, and a second row of 32 mux2t1s (G_OUT_REV) reverses the result back. That costs 64 extra muxes instead of a second shifter. The fill bit is automatically 0 for sll because s_Fill is ANDed with i_Right.

[Part 3.3.1.(d)] Describe how the execution of the different shifting operations corresponds to the QuestaSim waveforms in your writeup.



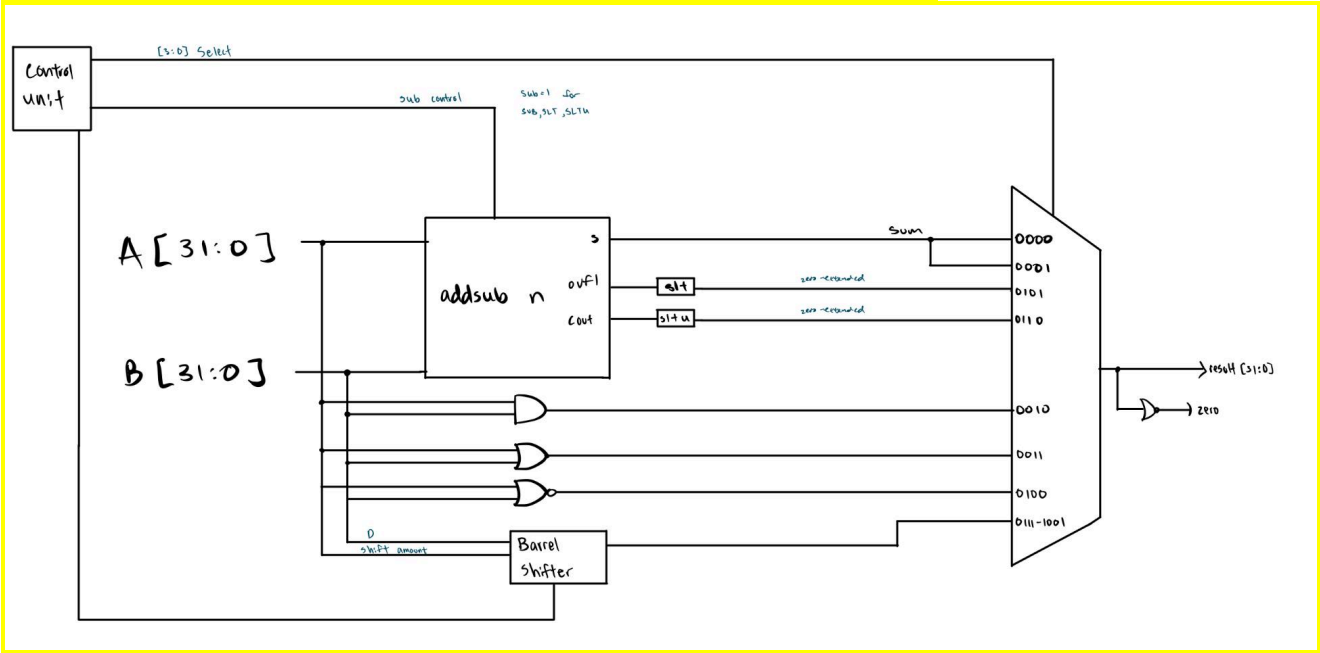
The tb_barrel_shifter testbench runs 12 cases, 10 ns each: sll from 0–50 ns, srl from 50–80 ns, and sra from 80–120 ns. s_Stage shows the value after each stage. The cases at 60 ns and 90 ns both shift 0xF0000000 right by 4 and differ only in the fill: srl gives 0x0F000000 and sra gives 0xFF000000. All 12 cases pass.

[Part 3.3.2.(a)] In your writeup, briefly describe your design approach, including any resources you used to choose or implement the design. Include at least one design decision you had to make.

Load Unit: selects the byte (using addr[1:0]) or halfword (using addr[1]) from the word returned by data memory, then sign or zero extends it with the Lab 2 extender_N, with i_Sign = not funct3[2]. This handles lb, lh, lbu, lhu, and passes lw through unchanged.
Imm Gen (immGen.vhd): dataflow. It builds all five immediates at once from the instruction bits (s_I, s_S, s_B, s_U, s_J) with concatenation and sign replication, for example s_B = (31 downto 13 => inst(31)) & inst(31) & inst(7) & inst(30 downto 25) & inst(11 downto 8) & '0', and a with-select on ImmType picks one. We did not reuse extender_N here: the B and J immediates are scattered across the instruction, so an extender would still need the same bit shuffling in front of it, and writing each format out directly matches the ISA tables and is easier to check.
Resources: the textbook (P&H RISC-V edition, Ch. 4 and the RISC-V reference card), lecture slides, and the RISC-V ISA specification for the immediate formats.
Design decision: our data memory is word addressed (addr[11:2]), so the low two address bits go to the load unit to pick the sub-word instead of going to memory. Another decision: the Lab 2 ALU "pass immediate" trick for lui would have broken slti/sltiu, so lui instead routes the immediate straight to the result mux (ResultSrc = 11).

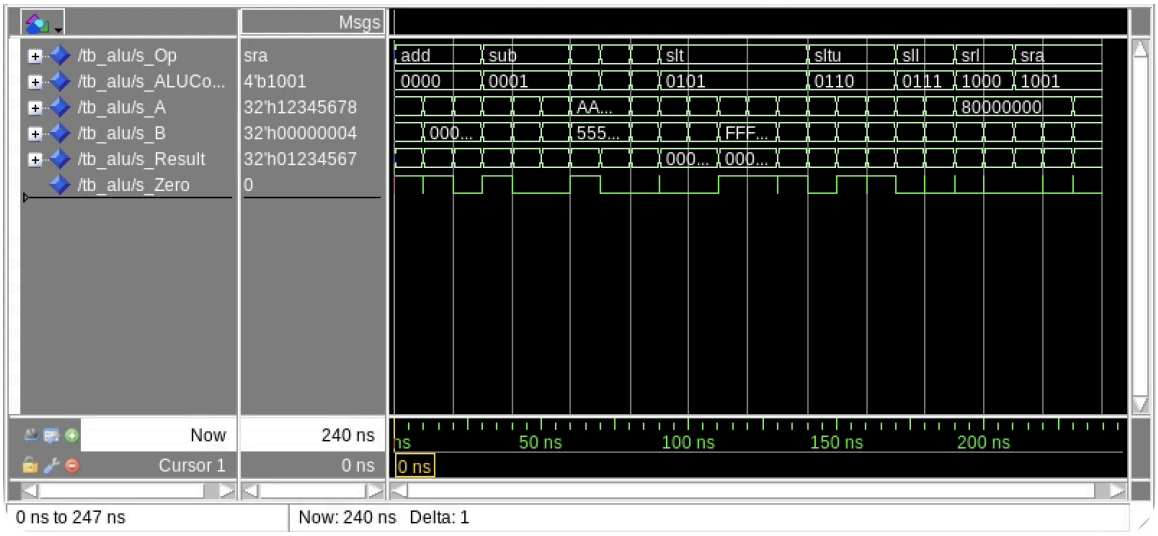
[Part 3.3.2.(b)] Describe how the execution of the different operations corresponds to the QuestaSim waveforms in your writeup.

[Part 3.3.3] Draw a simplified, high-level schematic for the 32-bit ALU. Consider the following questions: How is Zero calculated? How is `slt` implemented?



Zero is the NOR of all 32 result bits. For slt, the adder computes A − B and slt = S[31] XOR Overflow, so an overflow does not flip the result. For sltu, A < B exactly when A − B has no carry out, so sltu = NOT Cout.

[Part 3.3.5] Describe how the execution of the different operations corresponds to the QuestaSim waveforms in your writeup.



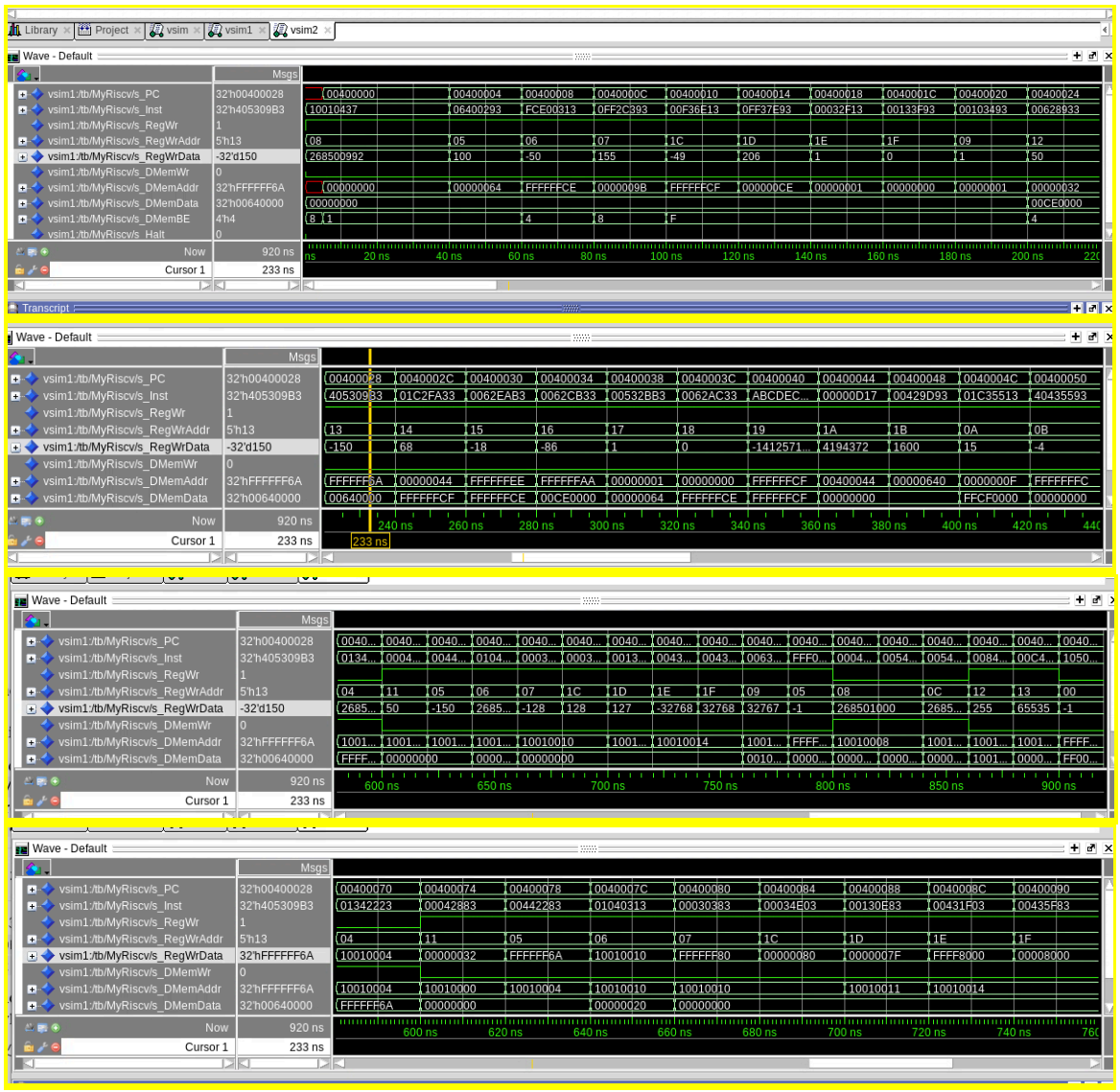
The tb_alu testbench runs 24 cases, 10 ns each, and s_Op shows the operation. Key cases: 0x7FFFFFFF + 1 wraps to 0x80000000; equal operands give 0 with Zero = 1 (the beq case); slt of 0x80000000 and 1 gives 1 even though A − B overflows; slt and sltu give different results for 5 and 0xFFFFFFFB; and B = 0x20 shifts by 0 because only B[4:0] is used. All cases pass.

[Part 3.3.8] justify why your test plan is comprehensive. Include waveforms that demonstrate your test programs functioning.

Each component was tested with its own testbench (ALU, barrel shifter, reg file, extender, fetch). At the processor level, we ran the provided programs plus our own: Proj1_lab2_test.s, Proj1_load_test.s, Proj1_store_test.s, and the three Part 4 programs. Together, these hit every required instruction at least once, every branch taken and not taken, shifts by immediate and by register, and signed vs. unsigned for slt, lb/lh, and sra/srl. The toolflow compares every register and memory write against RARS, so a pass means the whole trace matched. Everything passes with CPI 1.0. One catch: the processor halts on wfi, but RARS doesn't, so wfi has to be the last instruction in every program. Waveforms are in Part 4.

[Part 4] In your writeup, show the QuestaSim output for each of the following tests, and provide a discussion of result correctness. It may be helpful to also annotate the waveforms directly.

[Part 4.a] Create a test application that makes use of every required arithmetic/logical instruction at least once. The application need not perform any particularly useful task, but it should demonstrate the full functionality of the processor (e.g., sequences of many instructions executed sequentially, 1 per cycle while data written into registers can be effectively retrieved and used by later instructions). Name this file Proj1_base_test.s.



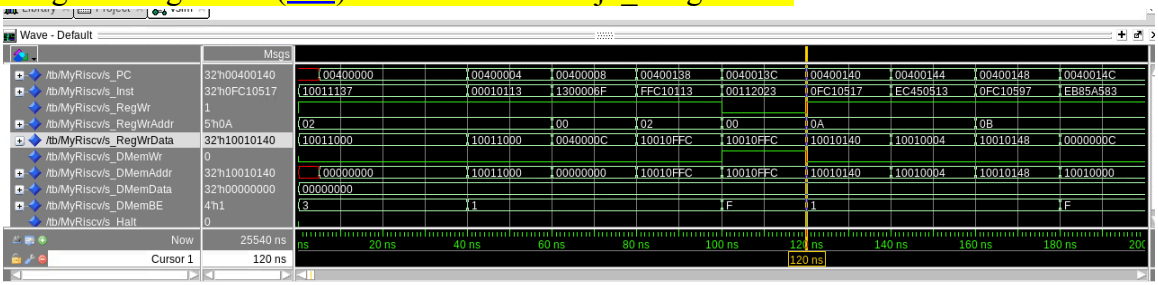
Proj1_base_test.s is 45 instructions and uses every required arithmetic, logic, shift, and memory instruction. Expected values are in the comments. Figure 5 shows the R-type section: sltiu gives 0 for -50 < 1 since it's unsigned, add/sub/and/or/xor give 50, -150, 68, -18, -86, and auipc writes 0x400044, which matches its PC. Figure 6 shows the shifts: srl of -50 by 28 gives 15, but srai by 4 gives -4, and sll by 35 gives the same 800 as sll by 3 since only the low 5 bits are used. Figure 7 shows the loads: lb/lbu on 0x80 give -128 and 128, lh/lhu on 0x8000 give -32768 and 32768, and sb/sh write 0xFF and 0xFFFF into cleared words. All writes matched RARS.

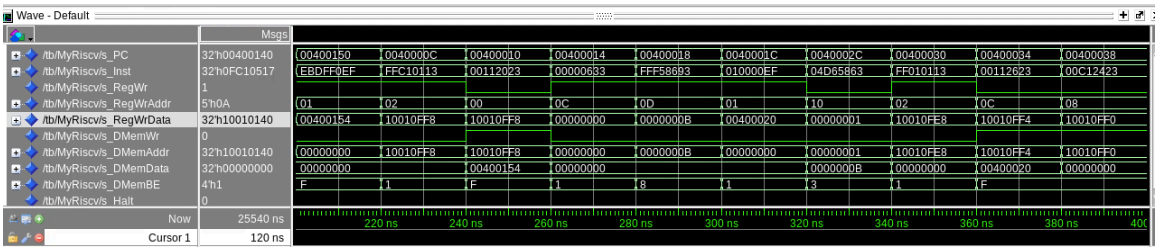
[Part 4.b] Create and test an application which uses each of the required control-flow instructions and has a call depth of at least 5 (i.e., the number of activation records on the stack is at least 4). Name this file Proj1_cf_test.s.

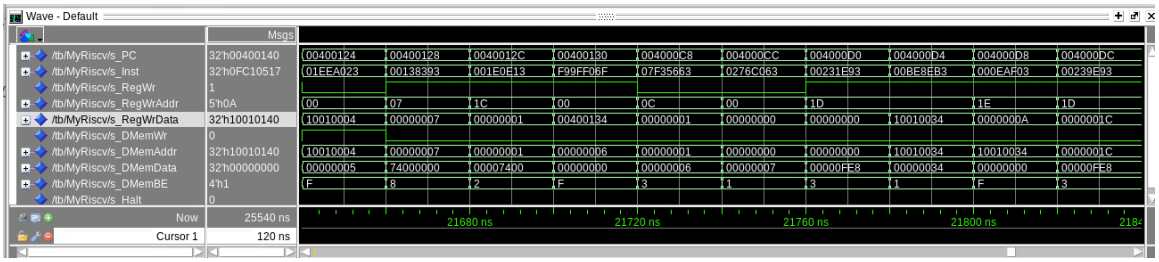


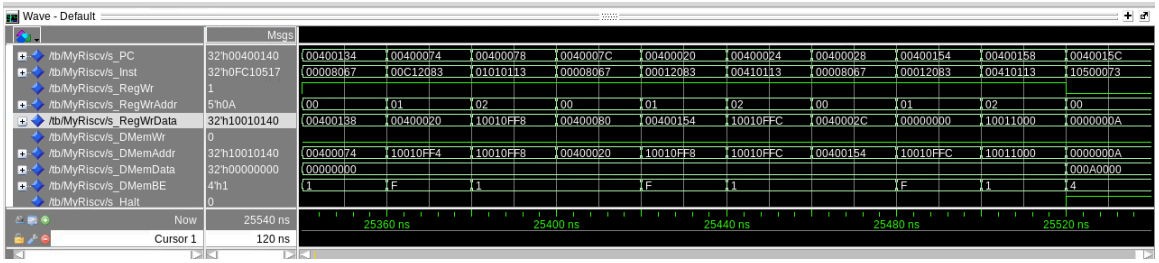
Proj1_cf_test.s is 145 instructions. It runs every branch (beq, bne, blt, bge, bltu, bgeu) once taken and once not taken, a backward branch loop, a recursive function 7 calls deep, and a jalr with a nonzero offset. Figure 8 shows the setup and first branches: beq t0,t1 is taken so the next instruction is skipped, beq t0,t2 is not taken so addi s1 runs and writes 1, then bne is taken. The rest of the branches work the same way and s1 ends at 63, which is stored to memory. Figure 9 shows the returns from the recursive calls: each one does lw ra, addi sp 8, jalr, then adds the saved value to a0, so sp climbs back up 8 per return and a0 goes 6, 10, 15 on the way to 21. The final results in memory are 63, 10, 21, 77 and every write matched RARS.

[Part 4.c] Create and test an application that sorts an array with *N* elements using the MergeSort algorithm (link). Name this file Proj1_mergesort.s.










```
------------------------------------------------
Testing file: proj/riscv/Proj1_mergesort.s
Rars simulation: pass
Modelsim simulation: pass
Test Result: pass
Rars Instructions: 1276
Processor Cycles: 1276
CPI:    1.0
Results in: output/Proj1_mergesort.s
------------------------------------------------
```

Proj1_mergesort.s follows the sort template: main sets sp, saves ra, loads a0 = array and a1 = array_size (12) and calls sort, which calls msort(lo = 0, hi = 11). It runs 1276 cycles at CPI 1.0. Figure 10 shows the recursion splitting the array: each msort call pushes a 16 byte frame (sp goes 0x10010FE8, FD8, FC8, FB8) and hi drops 11, 5, 2, 1 as the range halves. Figure 11 shows the last merge: the sorted left half 10, 11, 12, 65, 70, 89 is copied into tmp (0x10010034 to 0x10010048) and then merged with the right half 5, 7, 9, 45, 67, 90 back into array. The 90 is already in its final slot, so the merge stops when tmp runs out and does not rewrite it. Figure 12 is the trace of the final memory writes: 5, 7, 9, 10, 11, 12, 45, 65, 67, 70, 89 land in 0x10010004 through 0x1001002C, matching RARS. The array started as 65, 12, 10, 89, 11, 70, 67, 5, 9, 45, 90, 7 and ends as 5, 7, 9, 10, 11, 12, 45, 65, 67, 70, 89, 90.

[Part 5] Report the maximum frequency your processor can run at and determine what your critical path is. Draw this critical path on top of your top-level schematic from part 1. What components would you focus on to improve the frequency?

```
bash-5.1$ cd ~/CPRE381/Project_Pt1/project_pt1
head -30 timing.txt

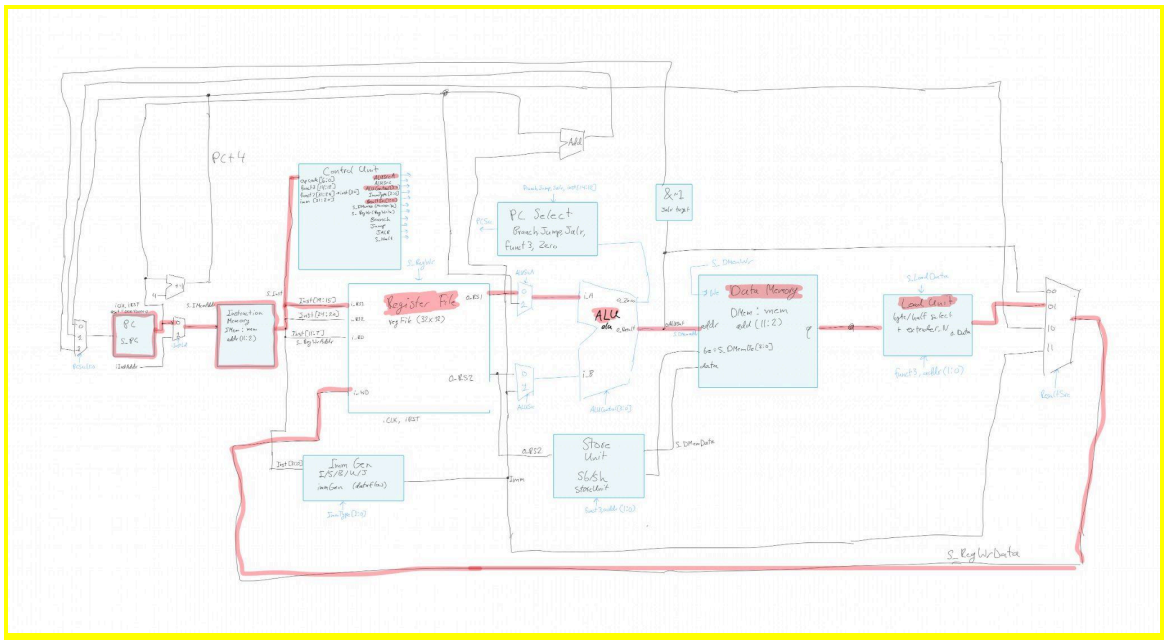
#
# CprE 381 toolflow Timing dump
#

FMax: 29.42mhz Clk Constraint: 20.00ns Slack: -13.99ns

The path is given below

 ====================================================================
 From Node    : fetchUnit:FETCH|pcReg:PC_REG|dffg:\G_BITS:7:G_ZERO:BIT_I|s_Q
 To Node      : regFile:REGS|reg_N:\G_REGISTERS:28:G_REG_REST:REGI|dffg:\G_NBit_REG:16:REGI|s_Q
 Launch Clock : iCLK
 Latch Clock  : iCLK
 Data Arrival Path:
 Total (ns)  Incr (ns)     Type  Element
 ==========  ========= ==  ====  ==================================
      0.000      0.000           launch edge time
      3.095      3.095  R        clock network delay
      3.327      0.232     uTco  fetchUnit:FETCH|pcReg:PC_REG|dffg:\G_BITS:7:G_ZERO:BIT_I|s_Q
      3.327      0.000 FF  CELL  FETCH|PC_REG|\G_BITS:7:G_ZERO:BIT_I|s_Q|q
      3.678      0.351 FF    IC  s_IMemAddr[7]~0|datad
      3.803      0.125 FF  CELL  s_IMemAddr[7]~0|combout
      5.662      1.859 FF    IC  IMem|Mux13~96|datab
      6.087      0.425 FF  CELL  IMem|Mux13~96|combout
      6.363      0.276 FF    IC  IMem|Mux13~97|dataa
      6.787      0.424 FF  CELL  IMem|Mux13~97|combout
      8.651      1.864 FF    IC  IMem|Mux13~100|datac
      8.932      0.281 FF  CELL  IMem|Mux13~100|combout
      9.160      0.228 FF    IC  IMem|Mux13~103|datad
bash-5.1$
```

.Synthesis gives FMax = 29.42 MHz (timing.txt). The critical path is the lw path, drawn in red on the schematic: PC_REG → s_IMemAddr mux (0.5 ns) → IMem read (8.2 ns) → control decode and MUX_ALUA (5.6 ns) → ALU adder and result mux (5.6 ns) → DMem read (8.4 ns) → load unit byte select and extender (2.0 ns) → MUX_RES (1.5 ns) → register file write (2.3 ns). Data arrives at 37.39 ns against 23.4 ns required for a 20 ns clock, so slack is -13.99 ns. To raise the frequency we would focus first on the two memories, which are 16.6 ns together because Quartus built them out of LUT mux trees rather than block RAM, then on the ALU, where the 32-bit ripple-carry addsub_n could be replaced by a carry-lookahead adder, and then on the decode logic in front of the ALU operand mux. Beyond that the single-cycle design is limited by fetch, execute, memory and write-back all being in series, which is what pipelining in Part 2 addresses.


PC+4
Control Unit
PC Select
Branch, Jump, Jalr, Funct3, Zero
&-2
Register File
Reg File (32x32)
ALU
Data Memory
Dmem [word addr (11:2)]
Load Unit
Store Unit
Imm Gen
I/S/B/U/J
S_RegWrData

```
------------------------------------------------
Testing file: proj/riscv/Proj1_base_test.s
Rars simulation: pass
Modelsim simulation: pass
Test Result: pass
Rars Instructions: 45
Processor Cycles: 45
CPI:   1.0
Results in: output/Proj1_base_test.s
------------------------------------------------
Testing file: proj/riscv/Proj1_cf_test.s
Rars simulation: pass
Modelsim simulation: pass
Test Result: pass
Rars Instructions: 145
Processor Cycles: 145
CPI:   1.0
Results in: output/Proj1_cf_test.s
------------------------------------------------
Testing file: proj/riscv/Proj1_lab2_test.s
Rars simulation: pass
Modelsim simulation: pass
Test Result: pass
Rars Instructions: 6
Processor Cycles: 6
CPI:   1.0
Results in: output/Proj1_lab2_test.s
------------------------------------------------
Testing file: proj/riscv/Proj1_load_test.s
Rars simulation: pass
Modelsim simulation: pass
Test Result: pass
Rars Instructions: 25
Processor Cycles: 25
CPI:   1.0
Results in: output/Proj1_load_test.s
------------------------------------------------
Testing file: proj/riscv/Proj1_mergesort.s
Rars simulation: pass
Modelsim simulation: pass
Test Result: pass
Rars Instructions: 1276
Processor Cycles: 1276
CPI:   1.0
Results in: output/Proj1_mergesort.s
------------------------------------------------
Testing file: proj/riscv/Proj1_store_test.s
Rars simulation: pass
Modelsim simulation: pass
Test Result: pass
Rars Instructions: 21
Processor Cycles: 21
CPI:   1.0
Results in: output/Proj1_store_test.s
------------------------------------------------
Testing file: proj/riscv/simplebranch.s
Rars simulation: pass
Modelsim simulation: pass
Test Result: pass
Rars Instructions: 12
Processor Cycles: 12
CPI:   1.0
Results in: output/simplebranch.s
------------------------------------------------
```